DIA MES AÑO NOTA /

## Good Practices in Object Oriented Modeling

Conceptual modeling represents ideas about a system so people can undertand, analyze, plan, and communicate about it. It can be used throughout sofware development: analysis, desin, specification, implementation, and deplogmet UML includes different diagrams for different purposes: use case diagrams show who uses a system and their goals; class diagrams show classes and their relationships; interaction diagrams show how objects communicate and state charts show changes in a object's behavior.

Good practices:

- Organize sofware into layers with clear responsibilities and few dependencies between them.
- Prefer aggregation or composition over inheritance when both choices work; this can make it easier to change which objects work together.
- Program to interfaces or abstractions to reduce unnecessary depencies.
- Choose clear names: classes are usually named with nouns, and methods with verbs.
- Write executable unit tests to check code, find errors, and repeat tests consistently.

DÍA MES AÑO NOTA 7

# Introduction to UML

UML means Unified Modeling Language. It is a collection of visual languages used to describe object-oriented systems. UML supports incremental development: some details can remain incomplete while the problem is being explored. It can be used with different software development processes.

A model can have several views. A view is a group of diagrams that shows the system from a particular perspective:

- **Analysis:** helps us understand the problem and real-world concepts.
- **Specification or high-level design:** describes what the system should provide.
- **Implementation:** represents concrete software components.

Diagrams can also be grouped into:

- **Structural diagrams:** show system elements and their relationships, such as classes, objects, components, and deployment.
- **Behavioral diagrams:** show goals and actions, such as use cases, sequences, activities, and states.

A diagram should have a clear purpose, use a consistent level of detail, and be easy to read. Avoid unnecessary diagrams and crossing lines. Use both structural and behavioral diagrams when needed.

DIA MES AÑO NOTA /

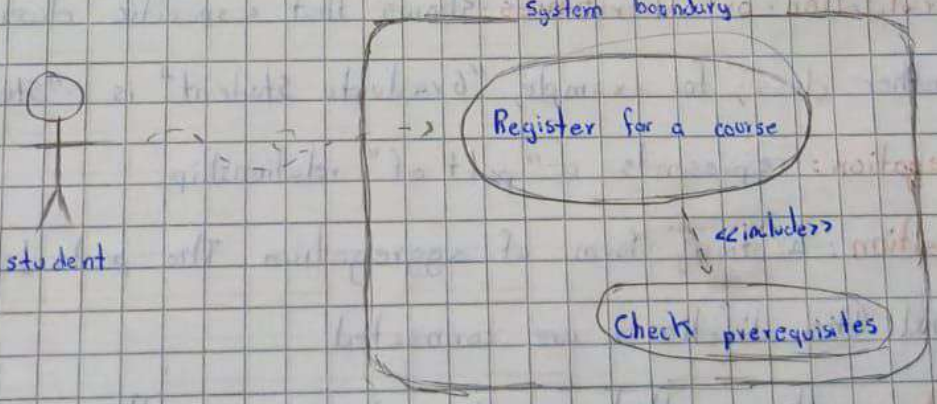
## Use Case Diagrams

An actor is a person, program, or device outside the system that interacts with it to achieve a goal. An actor represents a role, so one person can have several roles.

A use case describes a goal an actor wants to achieve with the system. It explains what the actor wants to accomplish and why. The detailed interaction steps explain how it happens. For example, "Register for a course" is a goal; selecting the course and submitting the request are interaction steps.

The main parts of a use case diagram are actors, use cases, relationships, and the system boundary.

- Association: connects an actor to a goal
- <<include>>: one use case requires another goal as part of its completion.
- <<extend>>: one use case adds a variation, often under a special condition. Its arrow points from the variation to the base use case.
- Generalization: a more specific actor or use case inherits features from a general one.



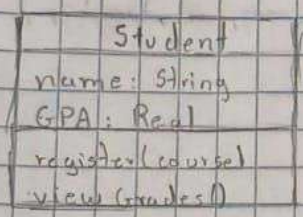

A well-structured diagram focuses on one part of the system, includes

only the elements needed to understand it, and has enough detail to communicate important goals.

## Class Diagrams

An object is an entity with identity, boundaries, and state or behavior. A class represents a group of objects that share characteristics. For example, "Student" can be a class, and "Ana" can be an object of that class.

A class is usually drawn as a rectangle with sections for its name, attributes and operations:




Attributes describe data or properties. Operations describe services or actions. UML visibility symbols include + for public, # for protected, and - for private.

Common relationships

- Association: connects related classes.
- Generalization: or inheritances: shows that a specific class is a type of another class; for example, "Graduate Student" is a "Student"
- Aggregation: represents a "part of" relationship.
- Composition: a strong form of aggregation. The parts depend on the whole, and their lifetimes are connected.
- Dependency: shows that one element uses another and may be

DÍA MES AÑO NOTA /

affected if it changes.

Multiplicity shows how many objects can be related: 1 means one; 0..1 means zero or one; * means many; and 1..* means one or more.

Multiplity is written besiden the appropriate class.

1.- Many - to - many association

Student ____ registers for ____ Course

A student may register for many courses;

a course may have many students

2.- Composition (whole - part relationship)

Order ◆——— Orderline

A class diagram should focus on a specific topic and keep a consistent perspective and leve of detail. Class diagrams alone may leave out important system behavior.

DIA MES AÑO NOTA /

# UML Quick Reference: Use Cases and Classes

During analysis, model the problem and the concepts understood by the people involved. Each diagram should communicate a main idea and remain readable.

For use cases, name actors with nouns and use clear goal names. Use include for a goal required by another use case; use extend for a variation or special condition. Notes can help record ideas while exploring the problem.

For classes, use clear singular names and name associations with phrases that explain the relationship. Use aggregation only when something is truly a part of something else. Otherwise, a named association may be clearer. Define and check multiplicities. During analysis, avoid deciding on attributes too early: an association may explain the relationship between concepts more clearly.